

Predictive Algorithms for Symmetric NAT Traversal Using Port Allocation Functions

Amit Kumar

amitmukhi@gmail.com

University Of South Carolina

Research Article

Keywords: Symmetric NAT, Communications NAT, VoIP, P2P, Traversal techniques

Posted Date: March 3rd, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-6128432/v1>

License:  This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: The authors declare no competing interests.

Predictive Algorithms for Symmetric NAT Traversal Using Port Allocation Functions

Amit Kumar

amitmukhi@gmail.com

Abstract—Network Address Translators (NAT) introduce significant challenges for peer-to-peer (P2P) communication services, such as Voice-over-IP (VoIP), by necessitating the use of central relay servers. This paper addresses the problem of traversing symmetric NATs, which are particularly problematic due to their complex port allocation rules. We propose several novel algorithms that exploit predictable port allocation functions to establish direct connections between hosts behind symmetric NATs. Our approach models the network environment as a stochastic process and evaluates the proposed algorithms under various network conditions. Simulation results demonstrate that our algorithms achieve high success rates, especially in low-workload networks, while minimizing system resource requirements. The theoretical insights gained from this work contribute to more efficient NAT traversal techniques, improving communication performance in security-sensitive P2P applications. Network Address Translators (NAT) introduce significant challenges for peer-to-peer (P2P) communication services, such as Voice-over-IP (VoIP), by necessitating the use of central relay servers. This paper addresses the problem of traversing symmetric NATs, which are particularly problematic due to their complex port allocation rules. We propose several novel algorithms that exploit predictable port allocation functions to establish direct connections between hosts behind symmetric NATs. Our approach models the network environment as a stochastic process and evaluates the proposed algorithms under various network conditions. Simulation results demonstrate that our algorithms achieve high success rates, especially in low-workload networks, while minimizing system resource requirements. The theoretical insights gained from this work contribute to more efficient NAT traversal techniques, improving communication performance in security-sensitive P2P applications.

I. INTRODUCTION

The proliferation of Network Address Translators (NATs) has surged in recent years, primarily as a measure to mitigate the rapid exhaustion of IPv4 address space. By allowing multiple devices within a private network to share a single public IP address, NATs serve as a practical solution to the limited availability of globally routable addresses. However, despite their benefits, NATs disrupt the fundamental end-to-end connectivity model of the Internet, posing significant challenges to peer-to-peer (P2P) communication. This shift inherently favors a centralized client-server model, making direct peer-to-peer interactions more complex and, in many cases, impractical without additional intervention.

Several applications rely heavily on P2P connectivity, with Voice-over-IP (VoIP) being a notable example where direct communication between participants is paramount. The inability to establish a direct link often necessitates the use of relay

servers, such as those implemented through the TURN protocol (1), to facilitate communication. However, relaying traffic through centralized servers introduces multiple inefficiencies: increased latency, higher jitter, and reduced bandwidth, all of which degrade the overall quality of service (QoS). Additionally, relay servers act as single points of failure and must scale with rising network demands, making them resource-intensive and financially burdensome for service providers. Consequently, finding an effective method for establishing direct connections between NAT-restricted hosts is of critical importance.

From a security perspective, the reliance on relay servers presents another major concern. These centralized intermediaries become attractive targets for attackers seeking to eavesdrop, intercept, or manipulate communications. In large-scale deployments, particularly where symmetric NATs function as carrier-grade border firewalls, this security risk is amplified. The forced reliance on relay servers makes monitoring and potential data breaches significantly easier for adversaries. Therefore, overcoming NAT traversal barriers is not only a matter of optimizing performance but also an essential step in enhancing the privacy and security of peer-to-peer communication.

This paper focuses on the traversal of symmetric NATs, which are widely regarded as the most challenging type of NAT due to their strict and dynamic port allocation rules. Unlike other NAT variants, symmetric NATs assign external port mappings uniquely for each outbound connection, rendering traditional NAT traversal techniques ineffective. However, in many cases, the port allocation process follows a predictable function that can be exploited for establishing direct peer-to-peer connections.

In this work, we introduce a set of novel predictive algorithms that leverage these deterministic port assignment patterns to facilitate NAT traversal. Our proposed methods are designed to minimize reliance on central relay servers by enabling direct communication between hosts behind symmetric NATs. By modeling the network environment as a stochastic process, we analyze the effectiveness of these algorithms under diverse network conditions. Through extensive simulations, we demonstrate that our approach achieves high success rates, particularly in low-workload scenarios, while maintaining minimal computational overhead.

The findings presented in this paper contribute to the broader field of NAT traversal research by offering a new

perspective on overcoming the challenges posed by symmetric NATs. The proposed methods not only enhance connectivity for VoIP and other P2P applications but also improve overall system efficiency by reducing dependency on resource-intensive relay infrastructure. Furthermore, by mitigating security vulnerabilities associated with centralized relays, our approach fosters more resilient and private communication channels in NAT-restricted environments.

II. EXISTING TECHNIQUES

Various methods have been proposed for going through the asymmetric NATs, including the NAT traversal mechanisms in Internet Key Exchange (2) and Interactive Connectivity Establishment (ICE) (3). Although some NAT devices provide specialized interfaces—such as the NAT Port Mapping Protocol (4)—these solutions are not generally utilized and universally dependent on.

ICE has become the predominant method owing to its versatility in handling multiple NAT types; however, it is ineffective when symmetric NATs are encountered. ICE primarily employs STUN (5) and, in some cases, TURN (see (1)) for its operation.

Several algorithms addressing symmetric NAT traversal have been introduced. For instance, Takeda (6) proposed methods for traversing symmetric NATs, yet the specific case in which both clients reside behind symmetric NATs with a port-sensitive allocation function was not considered. It was noted that increasing the number of transmitted packets does not necessarily enhance the probability of establishing a new connection. In contrast, the work presented herein demonstrates that direct connection establishment under these conditions is indeed feasible.

RFC 5128 (7) provides an overview of the s NAT traversal techniques; however, it does not address the scenario discussed in this paper. Additionally, the study by Wei *et al.* (8) focuses on symmetric NAT traversal using an alternative method that requires modification of the Time To Live (TTL) field in IP packets—a process that typically demands elevated privileges. Moreover, Wang *et al.* (9) have proposed a NAT traversal algorithm that relies on altering the source port, whereas the algorithms described in this work maintain a constant source port.

The present study is concerned with the scenario where two parties, denoted as A and B, attempt to establish a direct connection while both are situated behind symmetric NATs (or cascades thereof) that employ a port-sensitive, predictable allocation function. Cases involving predictable allocation functions in other contexts have been adequately addressed in previous research.

III. NETWORK ADDRESS TRANSLATION

The notation adopted in this paper is defined as follows. A *socket* is defined as

$$SC = \mathbb{IP} \times \text{PORT},$$

where $\mathbb{IP}$ denotes the set of all possible IP addresses and PORT denotes the set of all possible ports, with $\text{PORT} = \{1, 2, \dots, 65535\}$. A *socket pair* is defined as

$$SP = \mathbb{IP} \times \text{PORT} \times \mathbb{IP} \times \text{PORT}.$$

a) *NAT and Communication.*: NAT functions as a bridge between two distinct networks, commonly referred to as the internal and external networks, which are connected via corresponding NAT interfaces. Internal hosts are assigned addresses $\mathbb{IP}_{in} \in \mathbb{IP}_{in} \subseteq \mathbb{IP}$, while external hosts are assigned addresses $\mathbb{IP}_{ex} \in \mathbb{IP}_{ex} \subseteq \mathbb{IP}$, with the constraint that $\mathbb{IP}_{in} \cap \mathbb{IP}_{ex} = \emptyset$. Unless stated otherwise, the external NAT address is denoted by $\mathbb{IP}_{nat} \in \mathbb{IP}_{ex}$, under the assumption that the NAT employs a single external address for simplicity.

Packets received on the external interface are associated with a socket pair of the form

$$\{\mathbb{IP}_{ex}, \text{PORT}_{ex}, \mathbb{IP}_{nat}, \text{PORT}_{nat}\},$$

whereas packets received on the internal interface are associated with a socket pair of the form

$$\{\mathbb{IP}_{in}, \text{PORT}_{in}, \mathbb{IP}_{ex}, \text{PORT}_{ex}\}.$$

IV. UDP HOLE PUNCHING WITH STUN

The technique of *UDP hole punching* is employed by all the algorithms described in this paper. Due to NAT restrictions, direct packet delivery to an internal host is not possible, and communication must be initiated from within the internal network¹. This constraint pos

V. THEORETICAL NETWORK MODEL

To evaluate the proposed algorithms under varying conditions (e.g., different workloads and numbers of clients), the network behavior is modeled as a stochastic process. In particular, a Poisson process $\{N(t), t \geq 0\}$ is adopted, as it is commonly used in queueing theory to represent the arrival of requests to a server (10).

Since the key idea behind NAT traversal is to predict the next external port allocated by the NAT, it is essential to model the evolution of the NAT's internal state—that is, the number of external ports allocated over time. The workload, represented by the arrival rate, is therefore a critical parameter.

New connection events (i.e., new port allocations) are modeled as a time-homogeneous Poisson process $Po(\lambda)$ on the internal network. By varying λ , different network loads can be simulated, and the performance of the algorithms can be evaluated under various conditions.

Let $N(t)$ denote the number of new port allocations during the time interval $[0, t]$, with

$$N(t) \sim Po(\lambda t),$$

so that

$$P[N(t) = n] = \frac{(\lambda t)^n}{n!} e^{-\lambda t}.$$

¹This analysis does not consider pre-configured port forwarding, DMZ settings, uPNP, etc.

An incremental NAT is assumed, with $\Delta = 1$, on both sides. When the NAT state is sampled at intervals of $T = 10$ ms (for example, by a STUN request), the state is modeled as a random process $\{C_i \in \mathbb{N} : C_i \geq i, i \geq 0\}$ defined by:

$$C_i = 0, \text{ if}$$

$i=0,$

$C_{i-1} + 1 + X_i, \text{ otherwise, with } X_i \sim \text{Po}(\lambda T).$

This may be rewritten as

$$C_i = i + \sum_{j=1}^i X_j,$$

with an expected value of

$$E[C_i] = i(1 + \lambda T).$$

a) *Port Pool Exhaustion.*: It is assumed that NAT allocations time out after 3 minutes—a common practice. This assumption limits λ to the interval $[0, 0.36]$, which corresponds to an average of up to 360 new connections per second. Beyond this limit, the NAT port pool would be exhausted rapidly.

VI. GENERIC ALGORITHM STRUCTURE

All the algorithms we consider share a common framework: they work in discrete time steps to maximize the chance of establishing a direct connection between two hosts. In every step, each party either attempts to create a new hole in its local NAT or predicts the remote NAT's port allocation.

SYMMETRIC NAT TRAVERSAL ALGORITHM

We define a symmetric NAT traversal algorithm as a function

$$\mathcal{A} : \mathcal{U} \times \mathcal{S} \times \mathcal{M} \rightarrow \mathcal{P}_{\text{src}} \times \mathcal{P}_{\text{dst}},$$

where:

- $\mathcal{U} = \{0, 1\}$ represents the two communicating parties.
- $\mathcal{S} = \{0, 1, \dots, n-1\}$ (with $n \in \mathbb{N}$) denotes the set of discrete steps.
- $\mathcal{M}$ is a set of environmental parameters, which we assume to be

$$\mathcal{M} = \{(\lambda, t_s) \mid \lambda > 0, t_s \geq 0\},$$

where t_s represents the duration of the silent period, and λ characterizes the network's load.

- $\mathcal{P}_{\text{src}} \subseteq \mathbb{N}$ and $\mathcal{P}_{\text{dst}} \subseteq \mathbb{N}$ denote the sets of valid source and destination ports, respectively.

In practice, when the algorithm is run, party $u \in \mathcal{U}$ at step s uses the pair $\mathcal{A}(u, s, (\lambda, t_s)) = (p_1, p_2)$ where p_1 is the chosen source port and p_2 is the destination port. For instance, writing

$$\mathcal{A}(0, 1, (\lambda, t_s)) = (2, 3)$$

indicates that party 0, at step 1, sends a packet from source port 2 targeting destination port 3 at the remote side.

a) *Time Scheduling.*: The process begins at time $t_0 = 0$ ms. Each subsequent step $s \in \mathcal{S}$ is executed at time

$$t_s = sT,$$

with a typical step interval of $T = 10$ ms.

Algorithm Log and Connection Success

For a host (say, A), we record an *algorithm log* $\mathcal{L}_A^A$ that consists of tuples

$$(p_1, p_2, p_3),$$

where:

- $(p_1, p_2) = \mathcal{A}(0, s, (\lambda, t_s))$ are the ports chosen at step s .
- p_3 is the NAT-assigned port for the outgoing packet. This allocation is modeled by the recurrence

$$p_3 = C_s = C_{s-1} + 1 + X,$$

where X is a random variable drawn from a Poisson distribution with parameter λT .

- A similar log, $\mathcal{L}_A^B$, is maintained for the other host.
- To ensure uniqueness, it is required that for any two distinct tuples (p_1, p_2, p_3) and (p'_1, p'_2, p'_3) in $\mathcal{L}_A^A$, we have $p_3 = p'_3$ if and only if $p_1 = p'_1$ and $p_2 = p'_2$.

[Successful Connection] A run of the algorithm $\mathcal{A}$ is deemed *successful* (i.e., a direct connection is established) if there exist records

$$(p_1, p_2, p_3) \in \mathcal{L}_A^A \quad \text{and} \quad (p'_1, p'_2, p'_3) \in \mathcal{L}_A^B$$

such that

$$p_2 = p'_3 \quad \text{and} \quad p'_2 = p_3.$$

The overall objective is to design the function $\mathcal{A}$ so that the probability of a successful match is maximized while keeping the number of steps as low as possible.

EXAMPLE ALGORITHMS

Algorithm I – Baby-Step, Giant-Step

This algorithm is effective under low network load conditions (e.g., $\lambda \leq 0.03$ with $T = 10$ ms). One party increments its scanning port slowly (baby steps), while the other uses larger increments (giant steps). Formally, we define:

$$\mathcal{A}(u, s) = \begin{cases} (0, s), & \text{if } u = 0, \\ (0, 2s), & \text{if } u = 1. \end{cases}$$

Here, the source port remains fixed at 0 for both, while the destination port is incremented by 1 or 2, depending on the party.

Algorithm II – Fixed Destination Port

In this approach, the algorithm varies the source port in every step while keeping the destination port constant. This strategy has been previously discussed in the literature (9). It is defined by

$$\mathcal{A}(u, s) = (s, \Delta_u),$$

where the constants Δ_0 and Δ_1 are predetermined during the initialization phase based on the network workload.

Algorithm III – Expected Port Value

When the parameters governing the NAT's port allocation process are known, the algorithm can predict the expected port value. The prediction is based on the initial silent period and the subsequent step dynamics:

$$\mathcal{A}(u, s, (\lambda, t_s)) = (0, \lambda t_s + s(1 + \lambda T)),$$

for all $u \in \mathcal{U}$ and $s \in \mathcal{S}$. At each step s , the algorithm attempts to use the predicted NAT port, $C_s = \lambda t_s + s(1 + \lambda T)$, which is expected to be near the actual allocation given the unimodal nature of the distribution.

VII. EVALUATION

a) Comparison of Algorithms I and II.: It was observed that Algorithm I offers several advantages over Algorithm II. First, Algorithm I achieves a high success rate for network loads in the range $\lambda \in [0, 0.035]$, which corresponds to approximately 35 new connections per second on average. Second, the algorithm can be terminated as soon as a connection is established, with the overhead scaling proportionally to λ . Third, by using a single source port, only one listening thread is required, thereby simplifying the implementation and reducing system resource demands². Finally, Algorithm I is λ -invariant in low-load conditions, eliminating the need for additional measurements.

b) Simulation Results.: Figure 1 presents the success rates of the algorithms as a function of λ , while Figure 2 illustrates the average number of steps required to establish a connection. The results indicate that Algorithm I is most effective in low-workload environments, achieving a 100% success rate for $\lambda \in [0, 0.035]$. Beyond this range, however, new connections are generated faster than the algorithm can progress, leading to failure.

Algorithm II is configured with a fixed $\Delta = 900$ to maximize success; however, its reliance on a fixed destination port renders it vulnerable if that port is already occupied. Conversely, Algorithm IV demonstrates high success rates in noisy environments but is highly sensitive to the estimation of λ . Rapid variations in λ (i.e., a non-homogeneous Poisson process) can diminish its practical utility. Furthermore, a closed-form expression for the coefficient function $C(\lambda T)$ that would optimize the success rate remains an open research question.

VIII. REAL DATA MODELING

Data collected from the university network using NetFlow probes were analyzed to validate the Poisson process model for new NAT connection allocations. A simulation of NAT behavior was performed by feeding the NetFlow data into our program. The data capture spanned one hour of network communication during two different periods: 03:00 (low peak) and 13:00 (high peak).

²The NAT is leveraged to multiplex multiple connections on one source port.

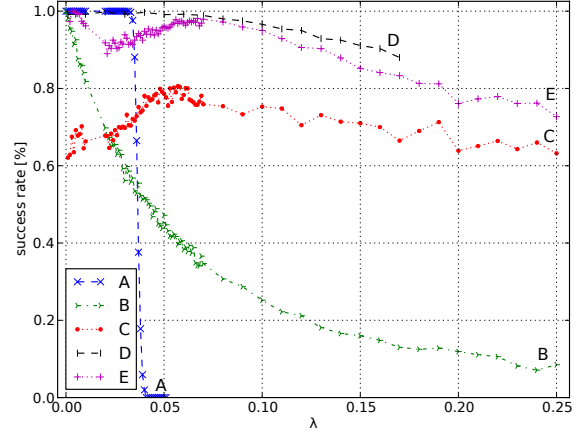


Fig. 1: Simulation evaluation of the algorithms with $T = 10$ ms. Each parameter setting was simulated over 1000 rounds. Algorithms A, B, and C correspond to Algorithms I, II, and III, respectively, while Algorithms D and E represent Algorithm IV with $C(\lambda T)$ optimized by exhaustive search (D) and approximated by a polynomial (E).

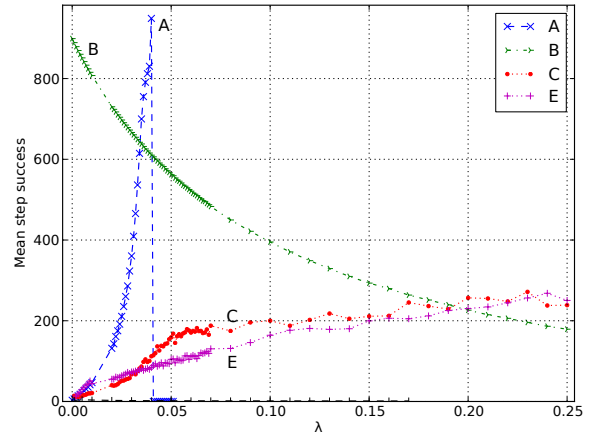


Fig. 2: Average number of steps required for successful connection establishment as a function of λ . The algorithms are denoted consistently with Figure 1.

To characterize the behavior of the NAT, we repeatedly measured the number of new connections established over fixed time intervals of $T = 10, 50$, and 100 ms. For each interval, we modeled the connection count using both the Poisson and Negative Binomial distributions. Parameter estimation for these models was carried out via Maximum Likelihood Estimation (MLE). In order to evaluate the adequacy of each model, we performed a Pearson χ^2 goodness-of-fit test at a significance level of $\alpha = 0.05$.

Table I summarizes the simulation results. For data set 13, collected during peak hours, it was found that when 100 samples (corresponding to a total time window of 1000 ms) are used, the process of creating new connections on the NAT can be modeled successfully by a homogeneous Poisson process (with the hypothesis rejected in only 5.191% of cases).

Set	T [ms]	Samples	Poisson r. [%]	NBinom r. [%]	$\bar{m}_{E[X]}$	$V[\bar{m}_{E[X]}]$	$\bar{m}_{V[X]}$
03	10	100	2.977	1.667	0.5121	0.0228	0.5321
		500	5.637	0.920	0.4851	0.0086	0.5146
		1000	10.294	1.579	0.4831	0.0068	0.5142
	50	100	6.543	2.276	2.3707	0.2373	2.9969
		500	25.180	7.194	2.3871	0.1053	3.1179
		1000	52.174	23.188	2.3920	0.0696	3.1576
	100	100	8.914	3.911	4.7263	0.7031	7.1496
		500	57.746	9.859	4.7668	0.2876	7.4667
		1000	5.191	1.824	2.6438	0.2695	3.0020
13	10	500	20.368	2.829	2.6579	0.1025	3.1495

TABLE I: Results from NAT simulation on real data captured from the university network. Columns represent the data set name, sampling time, number of samples collected, percentage of cases rejecting the Poisson hypothesis, percentage of cases rejecting the Negative Binomial hypothesis, sample mean of the estimated mean of port allocations, variance of the sample mean, and sample mean of the port variances.

However, when 500 samples are considered, the extended time window introduces significant variability in λ , leading to overdispersion. In this scenario, the Poisson hypothesis is rejected in 20.368% of cases, whereas the Negative Binomial model (better suited for overdispersed data) is rejected in only 2.829% of cases. The challenge that remains is how to incorporate the Negative Binomial model into effective NAT traversal strategies.

For data set 03, which represents a lower network workload, the sample means of the allocated ports remain stable, and the Poisson model is not rejected even for longer time intervals. Consequently, if the algorithm can accurately learn λ , the proposed methods should perform in practice as they do in simulation.

IX. DISCUSSION

It has been assumed for simplicity that the parameter λ is identical on both sides of the NAT. New challenges are anticipated if different parameters are permitted on each side, an aspect that has not yet been examined.

An open issue remains in the determination of a model that optimizes the primary objective of establishing a direct connection. Although a Poisson process was employed to model the network behavior during the initial phase, the approach requires the definition of a coefficient function $C(\lambda T)$, for which no exact analytical derivation is currently available. In addition, the method exhibits high sensitivity to parameter fluctuations, thereby limiting its practicality in real-world network environments.

A fixed time interval of $T = 10$ ms was assumed to minimize the rate at which the Poisson process evolves. It is recognized that such a short sending interval may lead to packet loss en route to the destination; however, the critical requirement is that packets reach the local NAT to generate the necessary allocation mapping. As suggested in (9), the algorithm can initially be executed with $T = 10$ ms to establish the mapping, and then re-run with a higher interval (e.g., $T = 100$ ms) to ensure that packets are delivered successfully using the mappings created in the prior round.

The success rate of the algorithm is heavily dependent on accurate measurement of the network properties in which the client operates. Such measurements can be obtained prior to connection establishment, as is commonly done in ICE,

thereby enabling the collection of additional information and the construction of a more precise network model.

Furthermore, it is noted that when multiple host pairs attempt to execute the algorithm simultaneously within the same network, the overall workload may be artificially increased. This deviation from the ideal Poisson distribution could render the proposed algorithms ineffective in such scenarios.

X. CONCLUSION

The objective of this work was to examine NAT traversal from a statistical perspective and to develop an algorithm that achieves a high success rate under low network workloads while being simple to implement and requiring minimal system resources. Although several algorithms have been proposed to address this problem, some open questions remain.

The network workload was modeled as a Poisson process, and this assumption was verified against real data through hypothesis testing. A Python-based application was developed to simulate NAT behavior, compute port distribution statistics, and evaluate the performance of the proposed algorithms under various parameter settings.

To further validate the proposed approach, an Android application was implemented using the baby-step giant-step algorithm for traversing a symmetric carrier-grade NAT employed by a mobile phone operator. The implementation performed as expected, thereby confirming the viability of the approach.

REFERENCES

- [1] R. Mahy, P. Matthews, and J. Rosenberg, "Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)," RFC 5766, IETF, Apr. 2010.
- [2] T. Kivinen, B. Swander, A. Huttunen, and V. Volpe, "Negotiation of NAT-Traversal in the IKE," RFC 3947, IETF, Jan. 2005.
- [3] J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal for Offer/Answer Protocols," RFC 5245, IETF, Apr. 2010.
- [4] S. Cheshire and M. Krochmal, "NAT Port Mapping Protocol (NAT-PMP)," RFC 6886, IETF, Apr. 2013.
- [5] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing, "Session Traversal Utilities for NAT (STUN)," RFC 5389, IETF, Oct. 2008.

- [6] Y. Takeda, "Symmetric NAT Traversal using STUN," Internet Draft, IETF, 2003. [Online]. Available: <http://tools.ietf.org/id/draft-takeda-symmetric-nat-traversal-00.txt>
- [7] P. Srisuresh, B. Ford, and D. Kegel, "State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs)," RFC 5128, IETF, Mar. 2008.
- [8] Y. Wei, D. Yamada, S. Yoshida, and S. Goto, "A New Method for Symmetric NAT Traversal in UDP and TCP," 2008. [Online]. Available: <http://www.goto.info.waseda.ac.jp/~wei/file/wei-apan-v10.pdf>
- [9] Y. Wang, Z. Lu, and J. Gu, "Research on symmetric NAT traversal in P2P applications," in *Proceedings of the International Multi-Conference on Computing in the Global Information Technology*, ser. ICCGI '06. Washington, DC, USA: IEEE Computer Society, 2006, pp. 59–65.
- [10] R. Nelson, *Probability, Stochastic Processes, and Queueing Theory: The Mathematics of Computer Performance Modeling*. New York, NY, USA: Springer-Verlag New York, Inc., 1995.